

RAPPORT

BUREAU D'ETUDES — SYSTÈMES CYBER-PHYSIQUES

Saad BENJELLOUN HARZIMI
Aurélien VERBEKE

TABLE DES MATIERES

1	Modèle physique	2
1.1	Modélisation	2
1.2	Résolution	2
2	TP2 : Modèle continu du pendule inversé	3
2.1	Sans capteur	3
2.2	Avec capteur	6
3	TP3 : Modèle du robot Lego	8
3.1	Modèle continu sans capteur	8
3.2	Modèle continu avec capteur	9
3.3	Modèle discret avec capteur	11
4	TP4 : Contrôle du robot Lego NXT	13

1 Modèle physique

1.1 Modélisation

On cherche à modéliser le comportement d'un pendule inversé contrôlé.

On commence par appliquer le principe fondamental de la dynamique sur le système :

$$ml^2\ddot{\alpha}(t) + mlg \sin(\alpha(t)) = u(t) \quad (1)$$

Avec $g = 9,81 \text{ m.s}^{-1}$ et $l = 10 \text{ cm}$ par exemple.

Soit, en prenant $x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} \alpha \\ \dot{\alpha} \end{pmatrix}$

On obtient :

$$(S) = \begin{cases} \dot{x}_1(t) = x_2(t) \\ \dot{x}_2(t) = \frac{g}{l} \sin(x_1(t)) - \frac{\cos(x_1(t))u(t)}{l} \\ x_1(0) = \alpha_0 \\ x_2(0) = \dot{\alpha}_0 \end{cases} \quad (2)$$

1.2 Résolution

On souhaite maintenant stabiliser le pendule au point d'équilibre $x_e = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

Pour faciliter la résolution des équations, on effectue un développement limité à l'ordre 1 :

$$(S') = \begin{cases} \dot{x}_1(t) = x_2(t) \\ \dot{x}_2(t) = \frac{g}{l} x_1(t) - \frac{u(t)}{l} \\ x_1(0) = \alpha_0 \\ x_2(0) = \dot{\alpha}_0 \end{cases} \quad (3)$$

Soit sous forme matricielle

$$(S') = \begin{cases} \dot{x} = Ax + B & A = \begin{pmatrix} 0 & 1 \\ \frac{g}{l} & 0 \end{pmatrix} \\ x(0) = \begin{pmatrix} \alpha_0 \\ \dot{\alpha}_0 \end{pmatrix} & B = \begin{pmatrix} 0 \\ -\frac{1}{l} \end{pmatrix} \end{cases} \quad (4)$$

On pose le contrôle $u(t) = Kx(t)$ avec $K = (k_1 \ k_2)$

On cherche alors des valeurs propres négatives de $A + BK$ pour contrôler asymptotiquement le système.

$$\begin{cases} \det(A + BK) > 0 \\ \text{tr}(A + BK) < 0 \end{cases} \iff \begin{cases} \frac{k_1}{l} - \frac{g}{l} > 0 \\ -\frac{k_2}{l} < 0 \end{cases} \iff \begin{cases} k_1 > g \\ k_2 > 0 \end{cases} \quad (5)$$

2 TP2 : Modèle continu du pendule inversé

2.1 Sans capteur

Lors de la première partie de ce TP, on s'intéresse à la simulation d'un pendule inversé contrôlé par un retour d'état sans capteur. Ce contrôle est défini par $u(t) = u_e + K(x(t) - x_e)$ avec $K = (k_1 \ k_2)$, u_e et x_e respectivement la consigne au point d'équilibre et le point d'équilibre en question.

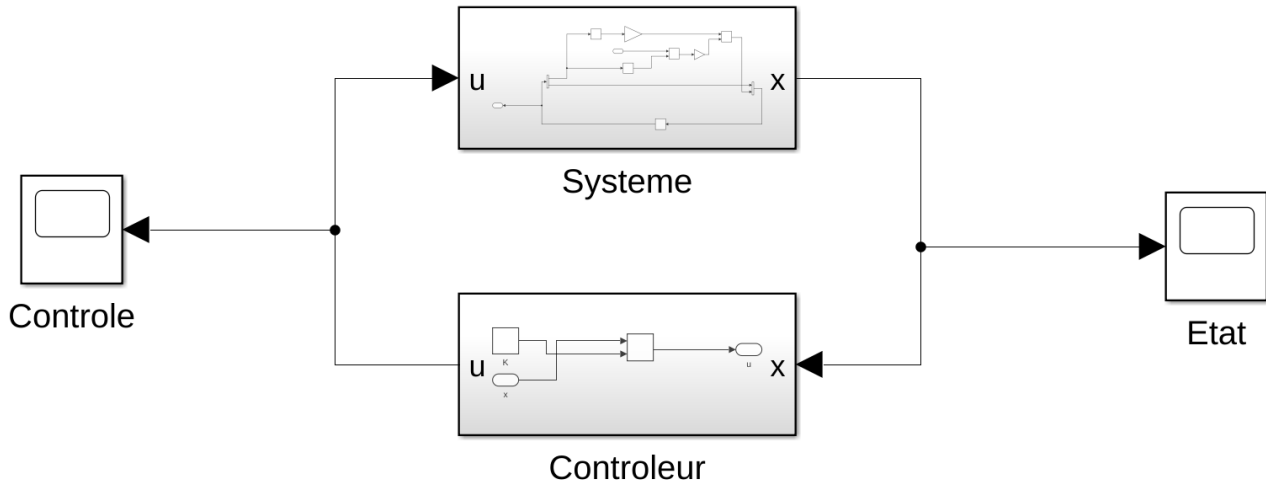
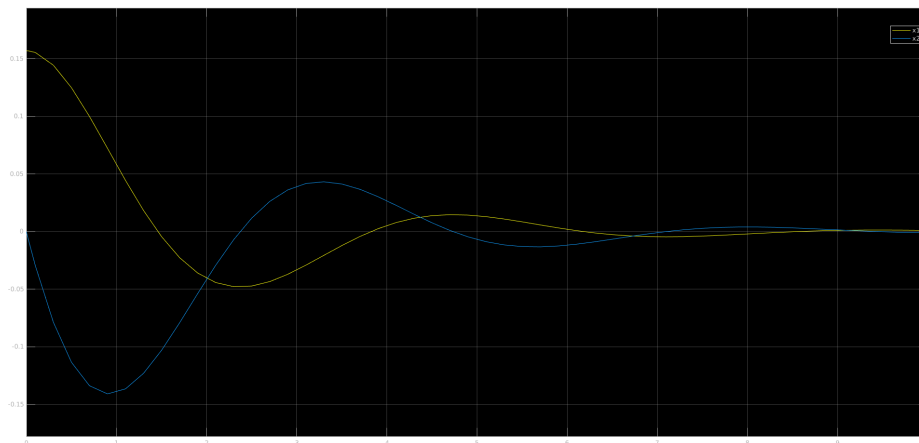


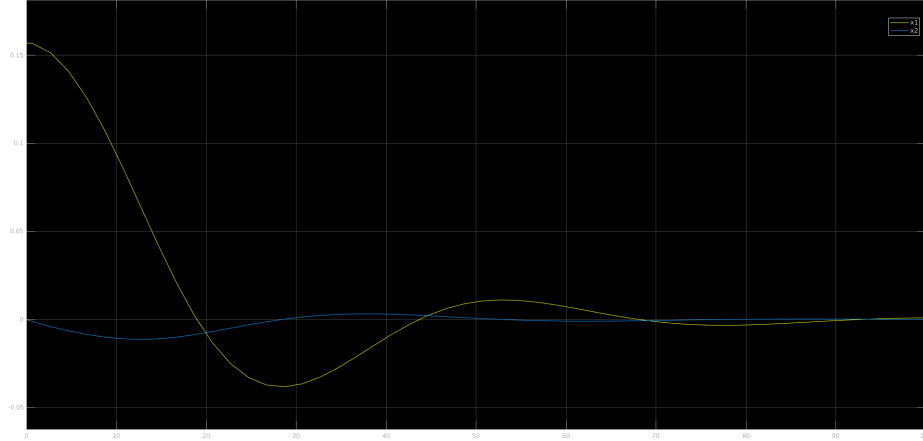
FIGURE 1 – Schéma du système contrôlé par retour d'état sur SIMULINK

— **Cas 1** : $x_0 = (\frac{\pi}{20} \ 0)$, $t_f = 10$, $K = (30 \ 10)$, intégrateur : ode45



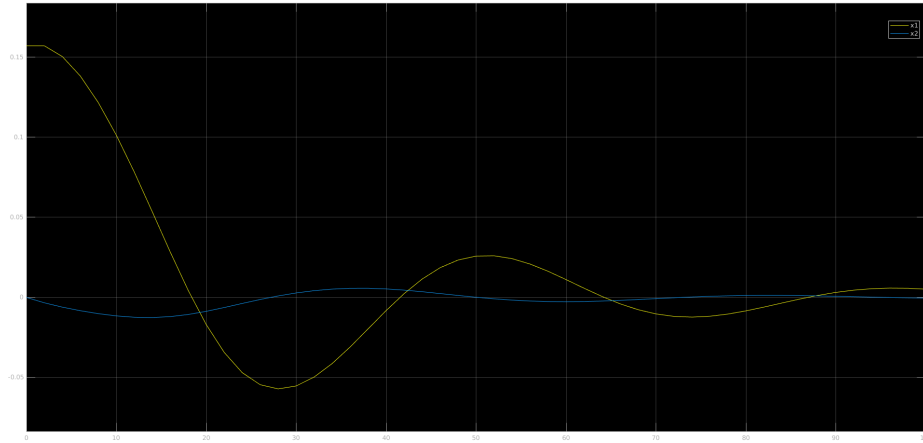
La simulation nous donne un premier bon résultat, même si le système met un peu de temps à converger (9 secondes).

— **Cas 2** : $x_0 = \left(\frac{\pi}{20} \ 0\right)$, $t_f = 100$, $K = \begin{pmatrix} 10 & 1 \end{pmatrix}$, intégrateur : ode45



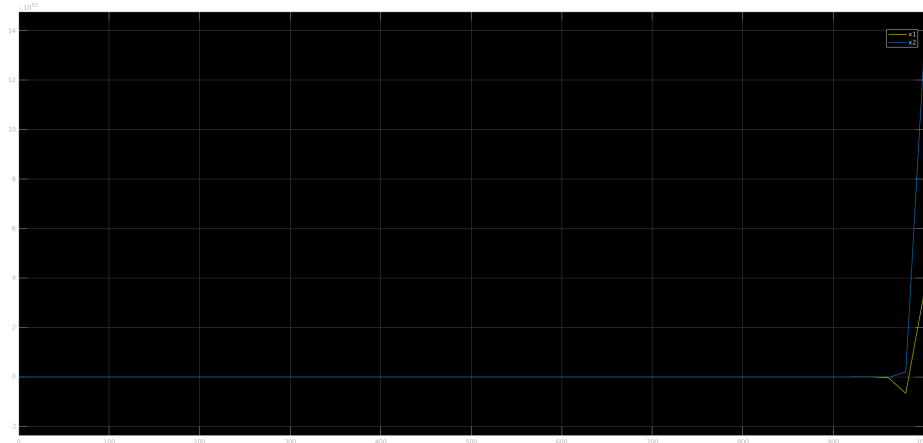
On constate en utilisant des valeurs moins importantes pour k_1 et k_2 et avec un temps assez long que le système converge bien vers le point d'équilibre $x_e(0,0)$, mais en plus de temps.

— **Cas 3** : $x_0 = \left(\frac{\pi}{20} \ 0\right)$, $t_f = 100$, $K = \begin{pmatrix} 10 & 1 \end{pmatrix}$, intégrateur : Euler, ode1



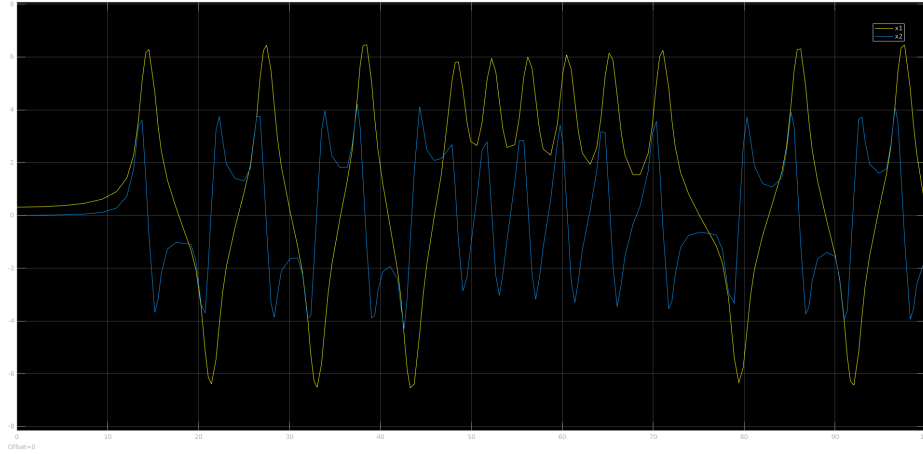
On remarque que l'intégrateur `ode1` nécessite un peu plus de temps pour être asymptotiquement contrôlé, contrairement à l'intégrateur `ode45`. Cependant, `ode1` fournit quand même une solution convergente qui se rapproche de celle donnée précédemment par l'intégrateur `ode45`.

— **Cas 4** : $x_0 = \left(\frac{\pi}{20} \ 0\right)$, $t_f = 1000$, $K = \begin{pmatrix} 10 & 1 \end{pmatrix}$, intégrateur : Euler, ode1



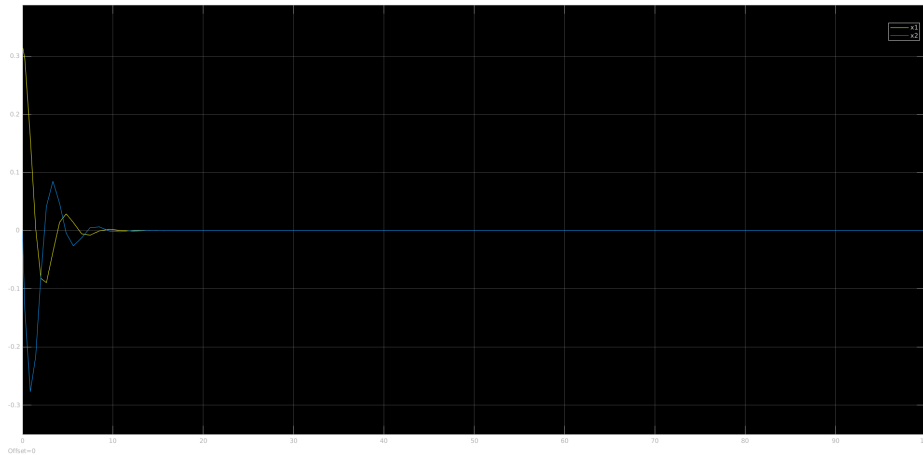
On constate ici que le système diverge : il semble qu'il y ait une erreur causée par l'intégrateur `ode1` pour un intervalle de temps trop important $t_f = 1000$.

— **Cas 5** : $x_0 = (\frac{\pi}{10} \ 0)$, $t_f = 100$, $K = \begin{pmatrix} 10 & 1 \end{pmatrix}$, intégrateur : ode45



En exploitant $x_0 = (\frac{\pi}{10} \ 0)$ et $K = \begin{pmatrix} 10 & 1 \end{pmatrix}$ comme paramètres, on remarque que la solution semble être erronée (le système peut en fait cycliser infiniment). Il n'est donc jamais contrôlé asymptotiquement. Il semble donc qu'il ne faille pas perturber le système trop fortement.

— **Cas 6** : $x_0 = (\frac{\pi}{10} \ 0)$, $t_f = 100$, $K = \begin{pmatrix} 30 & 10 \end{pmatrix}$, intégrateur : ode45



On reprend le cas précédent mais cette fois-ci en augmentant les coefficients de K : $(k_1 \ k_2) = (30 \ 10)$. On constate que le système est contrôlé au niveau asymptotique et converge.

Conclusion :

On peut déduire de cette première partie l'importance de choisir les paramètres k_1 et k_2 de façon à respecter les conditions établies lors du calcul théorique, à savoir $k_1 > g$ et $k_2 > 0$. On remarque aussi que ces deux valeurs doivent être assez grandes. Il existe cependant des erreurs qu'on a pu constater et qui proviennent des limites du modèle du pendule et des méthodes d'intégration.

2.2 Avec capteur

Dans cette partie du TP, on suppose qu'on n'a accès qu'à $\dot{\alpha}$. On va alors introduire deux sous-systèmes : un capteur et un prédicteur qui aura le rôle de reconstruire α en intégrant $\dot{\alpha}$. Pour cela, on réalise le schéma suivant :

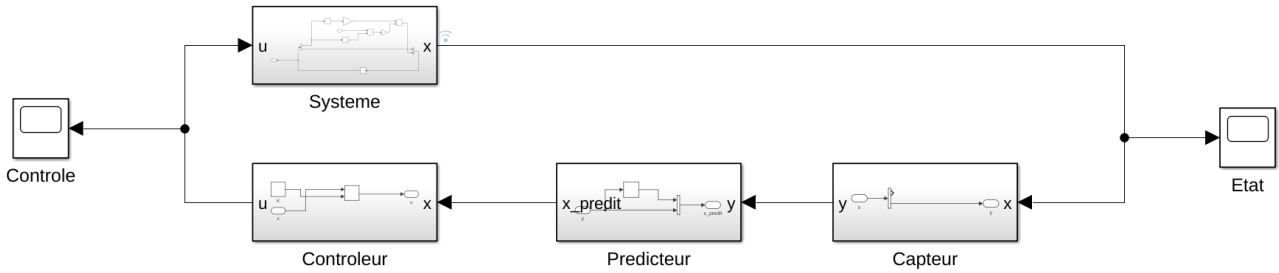


FIGURE 2 – Système contrôlé par retour d'état avec capteur et prédicteur sur SIMULINK

On effectue les simulations pour $x_0 = \left(\frac{\pi}{20} \ 0\right)$, $t_f = 100$ et $K = \begin{pmatrix} 10 & 1 \end{pmatrix}$

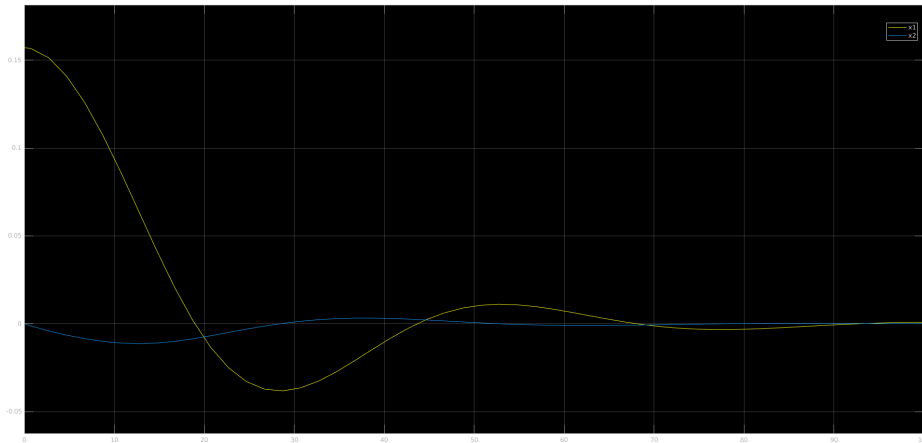


FIGURE 3 – Pas par défaut, intégrateur : ode45

On remarque que cette figure ressemble bien à celle du cas 2 vu auparavant, ce qui confirme les bons fonctionnements du capteur et du prédicteur.

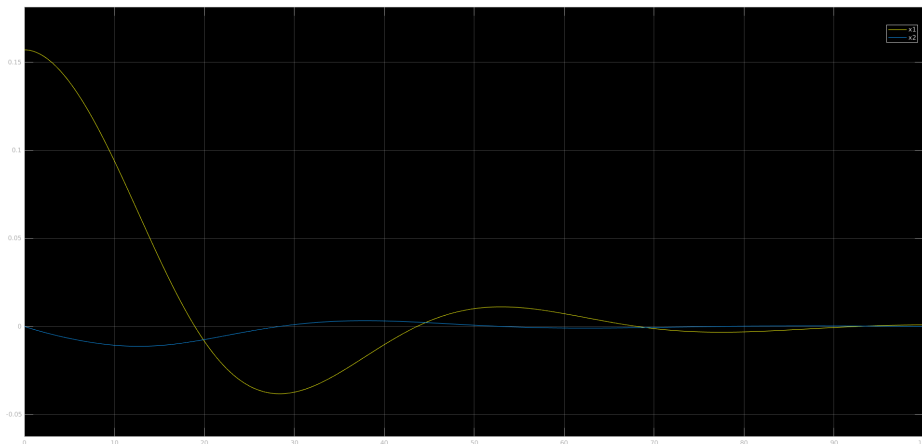


FIGURE 4 – $Pas = 0.001$, intégrateur : Euler, ode1

Pour un pas faible (0.001), on constate qu'on obtient une solution fournie par ode1 qui reste très proche du modèle en continu.

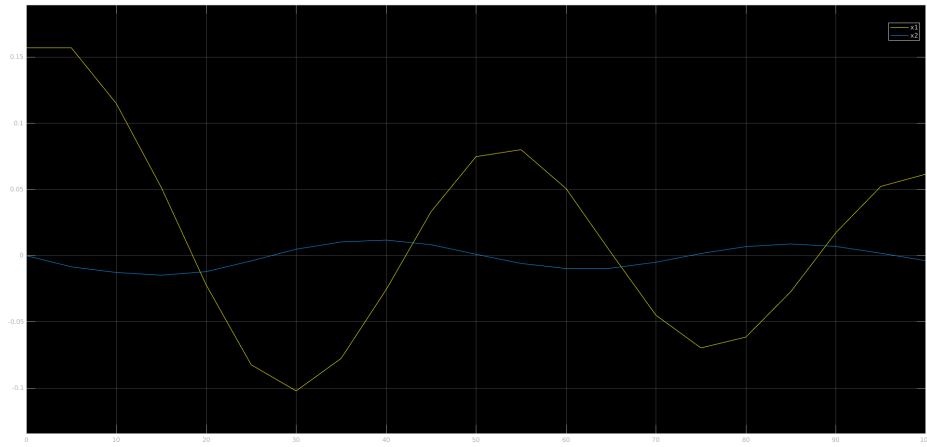


FIGURE 5 – $Pas = 5$, intégrateur : Euler, `ode1`

Concernant le choix $Pas = 5$, on remarque qu'on peut un peu reconnaître l'allure de la courbe. Cependant, on voit bien que le nombre de points est faible et que le calcul de la solution n'est pas efficace en précision.

Conclusion :

Si on voulait n'utiliser que l'intégrateur `ode1`, on devrait alors opter pour un pas faible afin d'avoir un bon équilibre entre la précision et le temps et l'efficacité de calcul de la solution numérique.

3 TP3 : Modèle du robot Lego

Le TP3 consiste à préparer le contrôle du robot Lego NXT.

La fonction régissant la physique du système est fournie.

Le contrôle est toujours implémenté sous la forme $u(t) = u_e + K(x(t) - x_e)$, où on prend $u_e = x_e = 0$, soit finalement $u(t) = Kx(t)$. Le contrôleur dans la figure ci-dessous est donc de conception assez simple.

On rappelle les 4 variables :

- θ : angle des roues
- $\dot{\theta}$: vitesse des roues
- ψ : angle du robot
- $\dot{\psi}$: vitesse de chute du robot

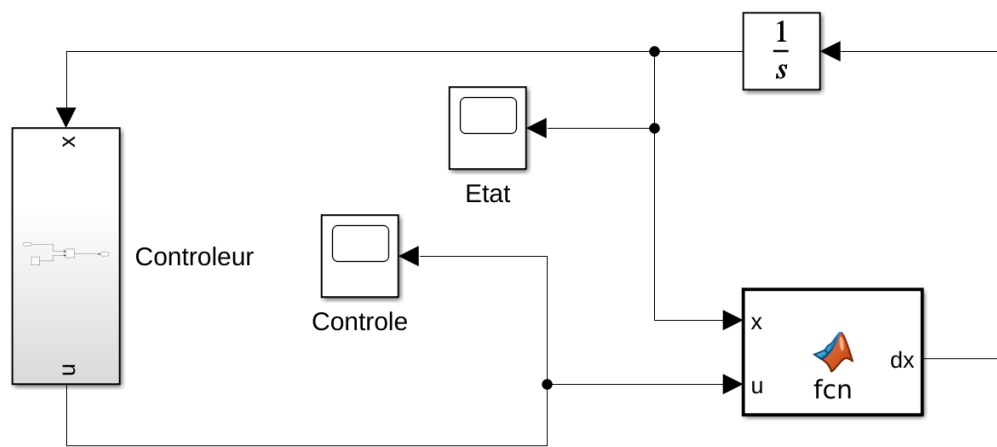
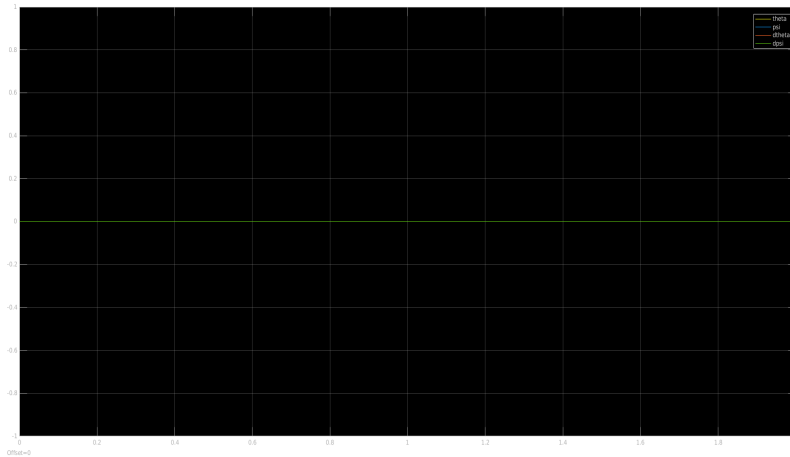


FIGURE 6 – Modèle SIMULINK du robot Lego NXT

3.1 Modèle continu sans capteur

Les simulations qui suivent sont toutes effectuées à l'aide de l'intégrateur ode45.

— **Cas 1** : $x_0 = (0 \ 0)$, $t_f = 2$



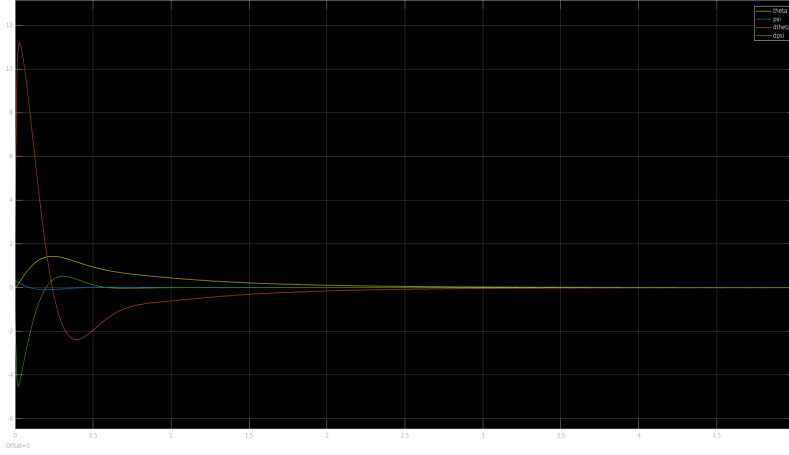
(a) Système



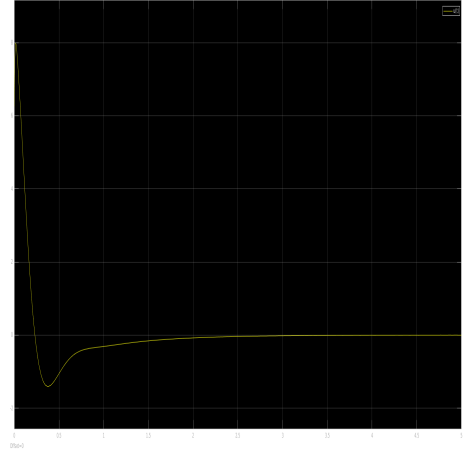
(b) Contrôleur

Le robot est ici placé dans son état stable, à la verticale. Il est donc normal qu'il soit contrôlé convenablement et reste à la verticale (sa position ainsi que celle des roues, et sa vitesse ainsi que celle des roues sont toutes nulles).

— **Cas 2** : $x_0 = (\frac{\pi}{10} \ 0)$, $t_f = 5$



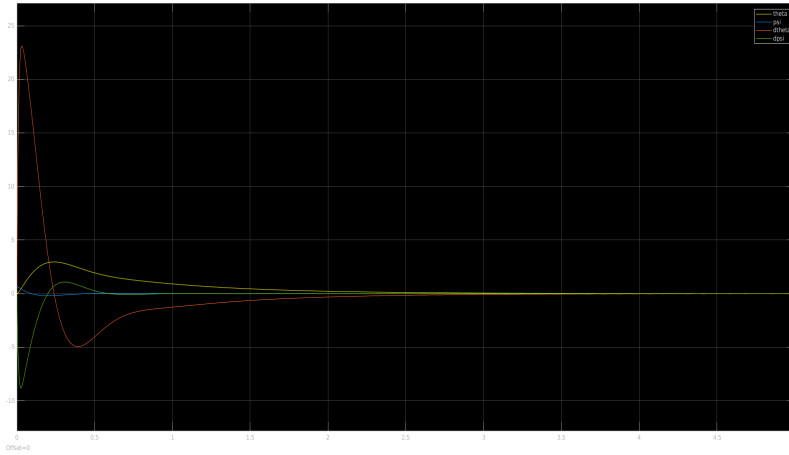
(a) Système



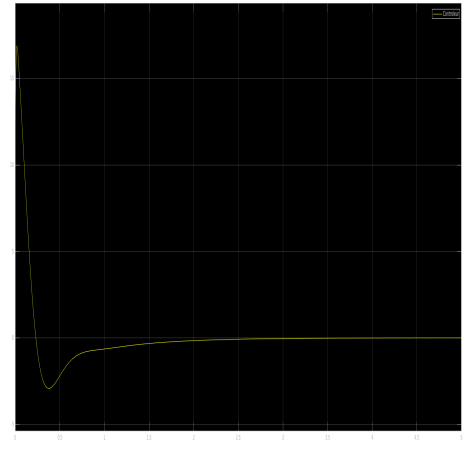
(b) Contrôleur

On impose ici une perturbation d'angle $\frac{\pi}{10}$ au robot. Le robot réussit à se stabiliser en environ 1 seconde puisque $\psi \rightarrow 0$ et $\dot{\psi} \rightarrow 0$.

— **Cas 3** : $x_0 = (\frac{\pi}{5} \ 0)$, $t_f = 5$



(a) Système



(b) Contrôleur

On augmente ici l'angle de la perturbation de l'angle du robot. Le robot converge cependant toujours aussi vite et bien vers sa position d'équilibre, même si l'angle commence à être élevé (35°). En regardant l'allure de la courbe du contrôleur, on observe que l'intensité du contrôle a simplement été accentuée.

3.2 Modèle continu avec capteur

On choisit ensuite de se rapprocher de la réalité en n'utilisant que les données qui seront à notre disposition : θ et $\dot{\psi}$. On prédit donc par dérivation et intégration les valeurs de $\dot{\theta}$ et ψ .

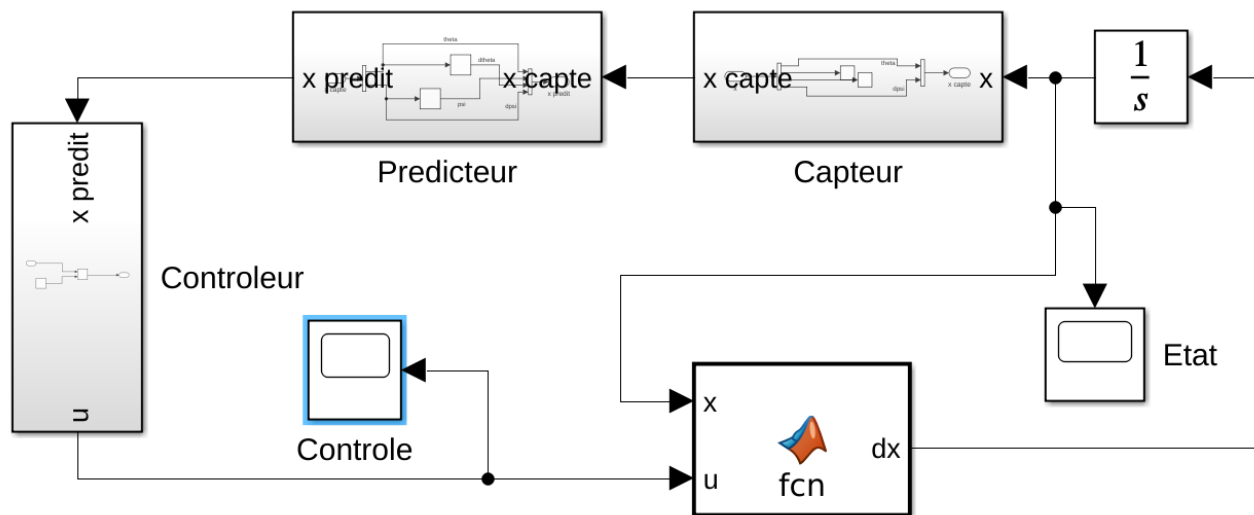
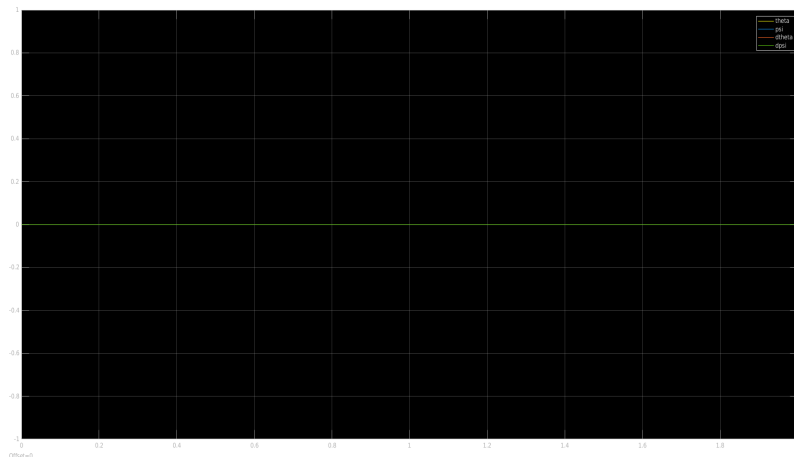


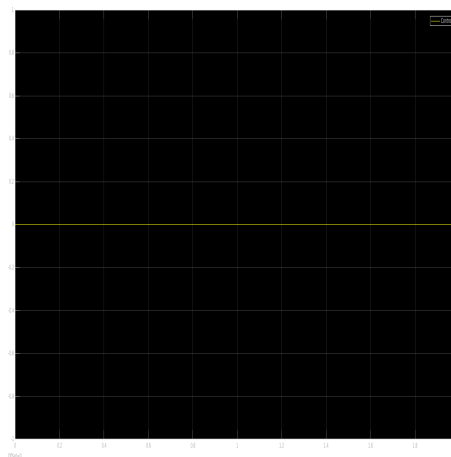
FIGURE 10 – Modèle SIMULINK avec capteur du robot Lego NXT

On répète alors les mêmes simulations.

— **Cas 1** : $x_0 = (0 \ 0)$, $t_f = 2$



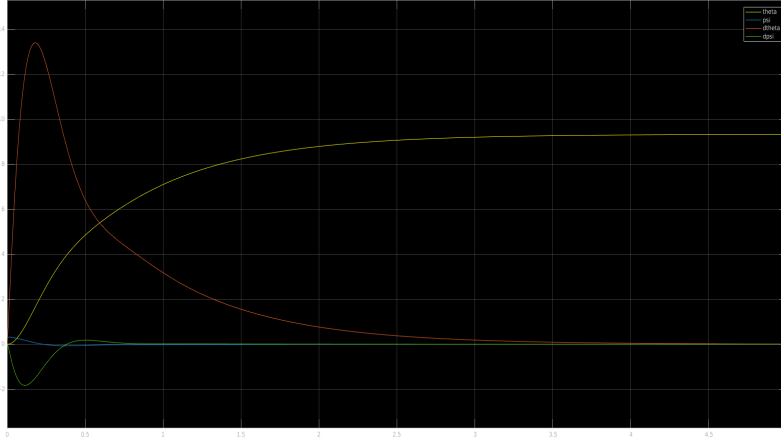
(a) Système



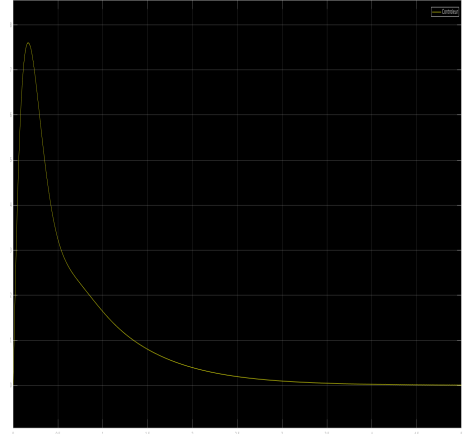
(b) Contrôleur

Le robot est ici toujours placé dans son état stable, à la verticale. Le commentaire est donc le même que dans la situation sans capteur : il est normal qu'il soit contrôlé convenablement et reste à la verticale (sa position ainsi que celle des roues, et sa vitesse ainsi que celle des roues sont toutes nulles).

— **Cas 2** : $x_0 = (\frac{\pi}{10} \ 0)$, $t_f = 5$



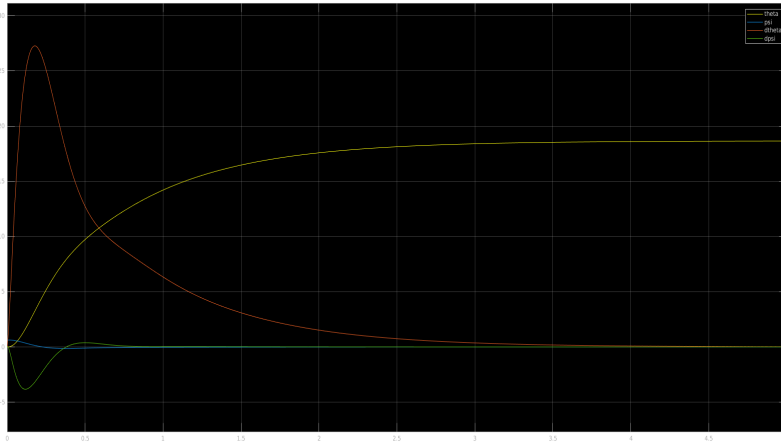
(a) Système



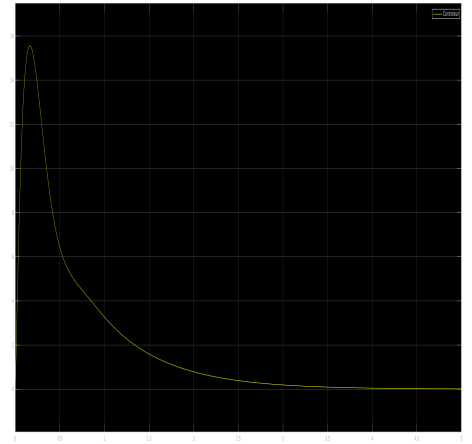
(b) Contrôleur

Dans cette situation, le robot se stabilise aussi vite en verticalité que sans capteur. Cependant, il ne se stabilise pas à sa position de départ : $\lim_{t \rightarrow +\infty}(\theta) \approx 6\pi$. Cela est certainement dû à l'absence du paramètre ψ .

— **Cas 3** : $x_0 = (\frac{\pi}{5} \ 0)$, $t_f = 5$



(a) Système

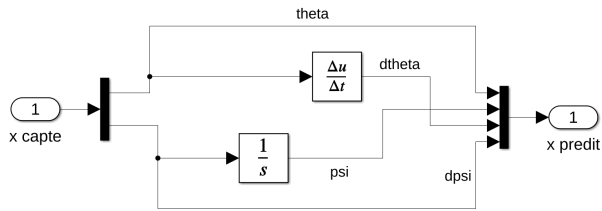


(b) Contrôleur

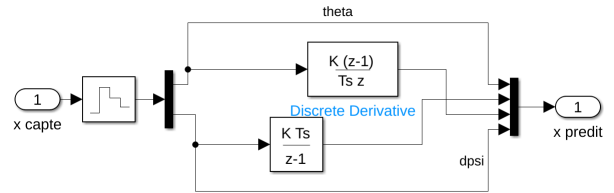
Le commentaire est ici le même que dans la situation sans capteur, avec les explications du cas avec capteur précédent.

3.3 Modèle discret avec capteur

Pour parfaire la simulation, on finit par discrétiser la mesure des paramètres θ et $\dot{\psi}$ ainsi que la prédiction de $\dot{\theta}$ et ψ . En effet, ces actions ne seront pas faites continûment mais selon les vitesses d'horloge de l'électronique et de traitement du programme.



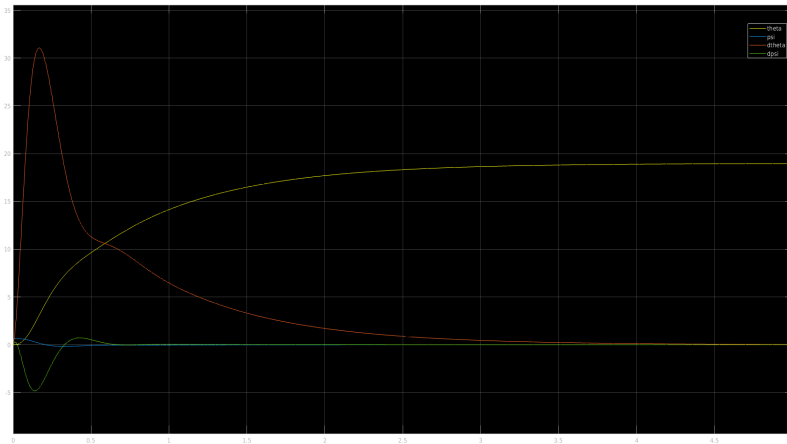
(a) Prédicteur continu



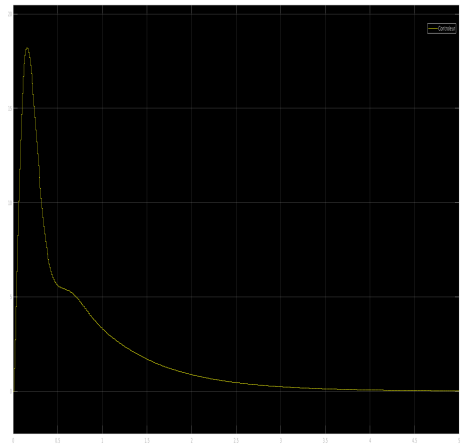
(b) Prédicteur discret

FIGURE 14 – Modèles SIMULINK des prédicteurs du capteur du robot Lego NXT

On répète la simulation de paramètres $x_0 = \left(\frac{\pi}{5} \ 0\right)$, $t_f = 5$, le pas de discrétisation est de 0,001 secondes :



(a) Système



(b) Contrôleur

Le robot parvient toujours aussi bien à se stabiliser, en appliquant cependant un contrôle plus fort.

Conclusion :

Même après ajout d'un capteur et sa discrétisation, on arrive à correctement simuler le comportement du robot en réponse à différentes perturbations.

4 TP4 : Contrôle du robot Lego NXT

On complète le code en C permettant de contrôler le robot (`nxtSegway.c`). Ce dernier fonctionnera parfaitement et nous surprendra même à descendre d'une table à une autre !

```
//...
void estimateur(float* y, float dt, float* x)
{
    float theta = .0;

    // Add psi_dot to get x
    x[3] = y[1];

    // Integrate psi_dot to get psi
    x[1] = x[3]*dt + x[1];

    // Theta = Theta_motor + psi
    theta = y[0] + x[1];

    // Derivate theta to get theta_dot
    x[2] = (theta - x[0]) / dt;

    x[0] = theta;
}

void controleur(float* xe, float ue, float setpoint, float dt,
               float* x, float* u)
{
    // Matrices of the controller
    float K[4] = {.67, 19.9055, 1.0747, 1.9614};

    // u = ue + K*(x-xe)
    *u = ue;
    for(unsigned char i=0 ; i<=3 ; i++)
        *u += K[i]*(x[i]-xe[i]);
}

//...

//...
case(CONTROL_MODE):
    dt = delta_t();
    y[0] = getMotorAngle();
    y[1] = getGyro(gyro_offset);
    estimateur(y, dt, x);
    controleur(xe, ue, setpoint, dt, x, &u);
    nxt_motors_set_command(u);
    break;
//...
```